

NSC Scanner — Final Forensic, Quant, Functional, Data and Production Audit Prompt

1. Your Role

Act as the combined:

- Principal Software Architect
- Senior Full-Stack Engineer
- Professional Indian Equity and Derivatives Trader
- Quantitative Trading-System Auditor
- Market-Data Engineer
- Broker API Integration Specialist
- Database and Accounting-Ledger Auditor
- Site Reliability Engineer
- Cybersecurity and Application-Security Auditor
- QA Automation Architect
- Production Release Auditor

You are auditing **NSC Scanner**, also referred to in earlier project materials as:

- NSESCANNER
- NSESCANNER-REPLIT
- Market Scanner by Dev
- Market Scanner

Project details:

- **Owner:** Devendra Sharma
- **Production website:** <https://marketscannerbydev.in>
- **Repository:** <https://github.com/devensharma87-creator/NSESCANNER-REPLIT>
- **Primary market:** Indian equity and derivatives markets
- **Authoritative timezone:** [Asia/Kolkata](#)
- **Operating model:** Owner-only market research, signal analytics and supervised paper trading
- **Live broker execution:** Not authorised and must remain disabled unless separately and explicitly approved

This is not a surface-level UI review. Conduct a complete forensic audit of the repository, database, APIs, deployed website, production behaviour, trading logic, calculations, data sources, background jobs, state machines, alerts, reports, security controls and operational reliability.

The primary objective is to identify everything that is:

- Correct
- Incorrect
- Incomplete
- Misleading
- Duplicated

- Dead or unused
- Inconsistently implemented
- Unverified
- Unsafe
- Fragile
- Over-engineered
- Under-tested
- Hardcoded
- Stale
- Synthetic
- Fabricated
- Improperly labelled
- Operationally unreliable
- Mathematically inaccurate
- Financially unsafe
- Missing from a professional-grade system

Do not assume that a feature works because code for it exists, a test passes, a prior report marked it complete, or the UI renders successfully. Prove actual end-to-end behaviour.

2. Audit-Only Authority

This assignment is for a **read-only audit and evidence-backed report**.

Do not:

- Modify production data.
- Change application code.
- create or close paper trades.
- Place real broker orders.
- Enable live trading.
- Change strategy thresholds.
- Send Telegram, WhatsApp, email or other alerts.
- Delete suspicious or polluted records.
- Clear logs or caches.
- Alter database schemas.
- Run destructive database commands.
- Rotate or expose credentials.
- “Fix while auditing” without first reporting the issue.
- Hide an issue by changing data or configuration.
- Treat a passing unit test as production proof.
- silently skip files, tabs, routes, jobs, databases or features.

You may run safe read-only inspections, local test suites, static analysis, builds, non-mutating API checks and controlled local reproductions.

If a test could mutate production state, send an alert, place an order or trigger an irreversible job, do not execute it. Document the blocked test and describe the safe method required to validate it.

Preserve all suspicious records as forensic evidence.

3. Required Inputs and Evidence

Before beginning, obtain and inspect all available material:

1. Complete repository, including all branches relevant to current production.
2. Exact production commit and deployment identifier.
3. Current `main` /working branch commit SHA.
4. Git status and uncommitted-change inventory.
5. Production website and preview/development deployment.
6. Route inventory and all navigation tabs.
7. Environment-variable name inventory, without printing secret values.
8. Database schema, migrations, constraints and indexes.
9. OpenAPI, Zod, TypeScript and database contracts.
10. Scheduled jobs, queues, cron jobs and background workers.
11. Provider adapters and broker integrations.
12. Test suites and test reports.
13. Logs, incident records and observability dashboards.
14. Existing audit reports, remediation plans and acceptance reports.
15. API/provider subscription limitations and rate limits.
16. Broker instrument masters and current exchange calendar.
17. Historical market-data coverage and raw-source evidence.
18. Alert configurations and delivery audit trails.
19. Deployment configuration, reverse proxy, domain and TLS configuration.
20. Backup, restore and disaster-recovery procedures.

Read all applicable project instruction files, including `AGENTS.md`, repository documentation and deployment instructions, before conducting the audit.

If access to any required item is unavailable, continue with the accessible evidence but mark the affected conclusion as **NOT VERIFIED**. Never convert lack of access into a pass.

4. Evidence and Truth Standards

Every conclusion must be classified as one of:

- **PROVEN CORRECT**
- **PROVEN DEFECTIVE**
- **PARTIALLY CORRECT**
- **IMPLEMENTED BUT NOT WIRED**
- **WIRED BUT NOT FUNCTIONAL**
- **TESTED LOCALLY ONLY**
- **TESTED IN PREVIEW ONLY**
- **PRODUCTION VERIFIED**
- **NOT TESTED**
- **NOT VERIFIABLE**

- **MISSING**
- **DUPLICATED**
- **DEAD CODE**
- **LEGACY FALLBACK**
- **SYNTHETIC OR MODELLED**
- **INFORMATIONAL ONLY**
- **UNSAFE FOR TRADING**

For every defect, provide:

- Unique issue ID
- Short title
- Severity
- Confidence
- Affected module
- Affected user journey
- Exact file and line or symbol
- Route/API/job/table involved
- Production evidence
- Reproduction steps
- Expected behaviour
- Actual behaviour
- Root cause
- Financial or operational impact
- Related dependencies
- Recommended correction
- Required regression tests
- Acceptance criteria
- Whether the issue blocks paper trading
- Whether the issue blocks future live trading
- Whether an earlier fix may regress

Do not use vague findings such as “could be improved” without explaining what is wrong, why it matters and how correctness will be proven.

Separate:

- Observed facts
- Code-derived conclusions
- Production-derived conclusions
- Inferences
- Hypotheses requiring further evidence

5. Severity Classification

Use this severity model consistently.

P0 — Stop-Ship / Critical

Examples:

- Incorrect trade entry or exit
- Trade created outside a valid market session
- Real order risk
- Incorrect capital or P&L
- Duplicate financial mutation
- Fabricated or stale data presented as live
- Authentication bypass
- Secret exposure
- Corrupted ledger
- Unrecoverable production-data loss
- Invalid signal represented as tradeable
- Production running unreviewed or unknown code

P1 — High

Examples:

- Wrong signal or market-state classification
- Broken risk control
- Wrong option selection
- Incorrect charges or lot size
- Missing execution truth
- Major data-source conflict
- Session, holiday or timezone error
- Broken canonical data path
- Material database/schema drift
- Provider outage causing misleading results

P2 — Medium

Examples:

- Feature partially broken
- Stale or incomplete report
- Incorrect UI explanation without direct financial mutation
- Missing reconciliation
- Performance failure affecting part of the market universe
- Incomplete observability or test coverage

P3 — Low

Examples:

- Minor UX inconsistency
- Non-critical documentation gap
- Maintainability issue

- Redundant code with no current behavioural impact

P4 — Improvement

A non-defective enhancement. Do not mix optional enhancements with actual bugs.

6. Mandatory Audit Method

Conduct the audit in the following sequence.

Phase 0 — Evidence Freeze and System Identification

Record:

- Audit date and time in IST
- Repository commit SHA
- Production deployment commit SHA
- Branch
- Dirty working-tree status
- Node, package manager, database and runtime versions
- Build commands
- Test commands
- Deployment configuration
- Production domain
- Preview domain
- Database environments
- Broker/provider environment
- Active feature flags
- Trading and paper-trading enablement flags
- Scheduled jobs and their effective timezone

Determine whether production actually corresponds to the repository being audited.

Compare:

- Source commit
- Built assets
- API version
- database migrations
- schema
- environment flags
- public frontend bundle
- health endpoint
- deployment metadata

Any inability to establish release identity is at least a P1 release-integrity finding.

Phase 1 — Complete Application Inventory

Programmatically enumerate:

- Frontend routes
- Backend routes
- Navigation tabs
- Hidden/owner-only routes
- Admin routes
- API endpoints
- WebSocket/SSE channels
- Scheduled jobs
- Queue workers
- Database tables
- Views and materialized views
- Provider adapters
- Broker adapters
- Strategy engines
- Calculators
- Signal generators
- Trade executors
- Report generators
- Alert senders
- Feature flags
- Legacy services
- Deprecated modules
- Unused components
- Duplicate implementations

Build a route-to-service-to-table-to-provider dependency map.

Do not rely only on the visible menu. Audit every reachable, hidden and backend-only feature.

For each feature, classify it as:

- Fully functional
- Partially functional
- UI shell only
- Backend only
- Unused
- Dead
- Duplicated
- Production-disabled
- Unsafe
- Not verifiable

Phase 2 — Architecture and Canonical Data Backbone

Verify whether the intended data pipeline is genuinely implemented:

Acquire

- Preserve raw evidence
- Normalize
- Validate
- Classify trust
- Publish through canonical facades
- Consume
- Audit
- Monitor

Confirm that trading engines and business modules do not independently call arbitrary external providers.

Audit for:

- Parallel data paths
- Direct provider calls from UI or strategy code
- Legacy provider fallbacks
- Duplicate quote services
- Duplicate instrument resolvers
- Different symbols for the same instrument
- Cache values overriding fresher canonical values
- Raw data used without validation
- Provider-specific types leaking into business logic
- Inconsistent timestamp parsing
- Conflicting definitions of "live," "fresh," "trusted" or "tradeable"
- Silent fallback to stale, delayed, modelled or synthetic data
- Different tabs using different prices for the same instrument and timestamp
- Business logic trusting display-only data
- Data received after the decision time being used retrospectively
- Look-ahead bias

For every displayed or calculated value, trace:

UI field

- API response field
- service/facade
- normalized record
- validation rule
- raw provider payload
- provider timestamp
- retrieval timestamp

The auditor must identify a single authoritative source and transformation chain for each important data class.

7. Provider and Data-Source Audit

Audit actual implementation and present-state availability of:

Zerodha Kite

Intended role:

- Primary trade-grade live quotes
- Instrument master
- Historical candles where permitted
- Holdings and positions
- Broker session state
- Future execution only if separately authorised

Verify:

- Authentication flow
- Session creation and persistence
- Token expiry
- Pre-market login readiness
- Automatic reconnect limitations
- Session-health checks
- Token encryption
- Logout and invalidation
- WebSocket reconnect behaviour
- Subscription management
- Rate-limit handling
- REST/WebSocket reconciliation
- Quote freshness
- Exchange timestamps
- instrument-token identity
- instrument-master refresh
- NSE/BSE resolution
- index resolution
- F&O contracts
- expiries
- strikes
- lot sizes
- tick sizes
- freeze quantities
- stale-session behaviour
- system-wide offline/degraded banner
- safe failure when the session is unavailable

A Kite failure must never silently become “market closed,” “zero setups,” “no opportunity,” or a valid flat-market conclusion.

Upstox

Intended role:

- Broker/data redundancy
- Independent verification
- Failover only under explicit trust rules

Verify whether it is:

- Integrated
- Partially integrated
- Configured but unused
- Not yet available
- Used inconsistently
- Capable of safe comparison/failover

Failover must not combine incompatible timestamps, symbols, candle definitions or units.

IndianAPI

Intended role may include:

- Fundamentals
- Company financial statements
- Corporate information
- News
- FII/DII information
- Participant-wise OI
- Market breadth
- Historical and reference information

Verify every claimed endpoint against the actual subscribed plan and API response.

Do not assume that a planned feature is implemented merely because credentials or configuration exist.

nselib and Official NSE/BSE Sources

Audit:

- Bhavcopy ingestion
- Exchange calendars
- corporate actions
- delivery data
- F&O contract files
- market reports
- symbol changes
- expiries
- holidays

- special sessions

Official exchange data should be used as the primary reference for exchange facts wherever legally and technically available.

Yahoo and Other Third-Party Sources

Apply strict source-honesty rules:

- Yahoo must not silently supply trade-grade prices.
- Delayed or derived analytics must be labelled.
- A total provider failure must show a degraded/unavailable state.
- Cached data must show its timestamp and age.
- Trading logic must not use informational-only prices.
- TradingView, Moneycontrol and similar websites may be secondary cross-checks, not canonical exchange authority.

Create a provider matrix covering:

- Data class
- Primary provider
- Secondary provider
- Permitted use
- Prohibited use
- Freshness limit
- Reconciliation tolerance
- Failure behaviour
- Cache policy
- Provenance fields
- Consumer modules

8. Data Quality, Provenance and Source Honesty

Verify that all financially relevant data contains appropriate metadata:

- Source
- Provider
- Exchange
- Instrument identifier
- Provider timestamp
- Exchange timestamp
- Received-at timestamp
- Normalized-at timestamp
- Published-at timestamp
- As-of timestamp
- Freshness
- Age
- Trust level
- Delayed/live/modelled status

- Validation result
- Failure reason
- Correlation/request ID

Audit for:

- Missing data represented as zero
- `null` converted to zero
- stale data represented as live
- delayed data represented as current
- modelled P&L represented as actual
- synthetic option prices represented as market premiums
- placeholder or mock data in production
- default objects that appear valid
- last-known-good data without age labels
- browser cache overriding API truth
- React Query stale caches
- service-worker cache contamination
- unit mismatches
- percentage/basis-point mismatches
- rupee/paise mismatches
- OI unit mismatch
- volume/OI confusion
- spot/futures/options price confusion
- NSE/BSE symbol collision
- timestamp/date/year corruption
- future-dated records
- duplicate candles
- missing candles
- out-of-order ticks
- impossible OHLC values
- negative or impossible volume/OI
- adjusted/unadjusted price mixing

Every tab must show a truthful state:

- Live and fresh
- Delayed
- Stale
- Degraded
- Informational only
- Modelled
- Synthetic
- Unavailable
- Market closed
- Provider offline
- Error

These states must not be interchangeable.

9. Trading Calendar, Time and Session Audit

Use `Asia/Kolkata` as the sole business timezone.

Verify:

- Regular NSE session: 09:15–15:30 IST
- Pre-open handling
- Exact 09:15 boundary
- Exact 15:30 boundary
- weekends
- official holidays
- special trading sessions
- Muhurat trading
- exceptional exchange closures
- clock drift
- UTC conversion
- database timezone
- frontend timezone
- backend timezone
- scheduler timezone
- report timezone
- candle bucket boundaries
- date rollover
- token-expiry time
- daylight-saving assumptions imported from non-Indian code

Do not treat a weekday as a trading day without checking the authoritative exchange calendar.

Test at minimum:

- Weekend
- Holiday
- Pre-open
- 09:14:59
- 09:15:00
- Intraday
- 15:29:59
- 15:30:00
- Post-close
- Special session
- Missing calendar
- Stale calendar
- Incorrect system clock
- UTC/IST conversion around midnight

Only a proven `marketOpen === false` state may show “Market is closed.”

Missing, stale or failed market-status data must show a neutral error/degraded state.

10. Tab-by-Tab Website Audit

Discover the actual tabs from the code and production UI. Audit every field, interaction, calculation, filter, table, chart, badge, message, button, empty state, loading state, error state and mobile layout.

The audit must include, where present:

Home / Market Pulse / Dashboard

Verify:

- Index values
- percentage changes
- previous close
- advances/declines
- breadth
- sector performance
- volatility
- FII/DII figures
- global-market values
- market status
- freshness labels
- sentiment summaries
- signal summaries
- portfolio summary
- paper-trading summary
- report timing
- news
- source attribution

Ensure market summaries are derived from current validated data and not stale narrative text.

Watchlist

Verify:

- Instrument resolution
- NSE/BSE identification
- LTP
- change and percentage
- OHLC
- volume
- freshness
- sorting
- filtering
- add/remove
- persistence
- duplicate prevention
- unavailable-symbol handling

- market-close behaviour
- provider outage behaviour
- concurrent updates

Scanner and Deep Scan

Verify:

- Full intended universe
- actual scanned count
- excluded count
- failed-symbol count
- cached count
- partial-universe labelling
- timeout behaviour
- pagination
- ranking
- filters
- indicators
- delivery percentage
- liquidity
- corporate-action adjustments
- source classification
- stale results
- scan timestamps
- duplicates
- survivorship bias

Investigate the historical condition where a full NSE scan degraded to approximately 198 cached rows or timed out.

Sector and Benchmark Modules

Verify:

- Constituent lists
- index weights
- sector mapping
- benchmark values
- breadth
- returns
- rebalance/corporate-action updates
- consistent price source

Portfolio Analyser

Verify:

- Holdings resolution
- NSE/BSE identity

- quantities
- average cost
- invested value
- current value
- realized P&L
- unrealized P&L
- day P&L
- absolute return
- XIRR/CAGR, where shown
- dividends
- corporate actions
- splits/bonuses
- charges
- missing-price handling
- duplicate holdings
- ETF/instrument classification
- rounding
- totals matching row-level values

Recheck historically problematic symbols such as:

- TRIDENT
- BDL
- CDSL
- BLS
- INDHOTEL
- ARE&M
- NSDL/BSE identity
- MAM150ETF or other potentially invalid instruments

Do not assume previous symbol-resolution fixes still work.

Charting

Verify:

- Symbol identity
- timeframe
- candle source
- candle completeness
- OHLC accuracy
- timezone
- session gaps
- indicators
- corporate-action adjustment
- zoom and range
- cache
- live candle formation
- closed-candle logic
- stale-data warning
- drawing persistence, if present

- consistency with scanner and signals

Option Chain / OI Lab

Verify:

- Underlying spot
- expiry
- strikes
- CE/PE mapping
- LTP
- bid/ask
- volume
- OI
- change in OI
- IV
- Greeks
- PCR
- max pain
- ATM selection
- support/resistance
- build-up classification
- timestamp alignment
- source
- freshness
- lot size
- missing contracts
- illiquid contracts
- crossed/invalid quotes
- stale chain detection

Re-prove the accepted Kite OI-unit correction.

Backtest Lab

Verify:

- No look-ahead bias
- no survivorship bias
- signal-time reproducibility
- realistic entry timing
- realistic exit timing
- real versus synthetic premium
- slippage
- bid/ask spread
- charges
- expiry selection
- strike selection
- liquidity
- missing-data exclusion
- session handling

- corporate-action adjustment
- partial-period handling
- reproducibility
- strategy/version identity
- parameter capture
- dataset fingerprint
- result determinism

Explicitly report whether results use:

- Real captured option premium
- Partially captured premium
- Black-Scholes modelled premium
- Synthetic delta proxy
- Spot-only proxy
- Unavailable pricing

Never combine these into one unlabeled performance number.

Daily Analysis and Reports

Verify:

- Pre-market report timing
- Post-market report timing
- IST correctness
- source timestamps
- market status
- no-data explanations
- signal summaries
- trade summaries
- skip reasons
- P&L reconciliation
- delivery evidence
- duplicate report prevention

Admin / Infrastructure / Health

Verify:

- Authentication
- owner-only access
- provider health
- last successful job
- last signal date
- trading-day gap
- data issue versus market-condition classification
- queue depth
- database health
- cache health
- schema version

- deployment version
 - job failure details
 - safe retry
 - observability
-

11. F&O Intraday Engine Audit

Audit the complete lifecycle:

```
Market/session readiness
→ Data readiness
→ Candidate discovery
→ Setup detection
→ Validation
→ Risk gating
→ Signal classification
→ Contract selection
→ Entry authorization
→ Paper-trade opening
→ Monitoring
→ Stop/target/trailing logic
→ Exit
→ Charges
→ Ledger
→ P&L
→ Reporting
→ Alerts
```

For NIFTY, BANKNIFTY, SENSEX and every supported F&O underlying, verify:

Market and Data Readiness

- Valid market session
- fresh Kite session
- fresh spot/index quote
- fresh and trusted option chain
- current instrument master
- valid expiry
- sufficient chain coverage
- no cross-session quote
- no delayed Yahoo price
- timestamp alignment
- provider readiness
- market-status truth

Setup and Strategy Logic

Inventory every setup and classify it as:

- Active
- Gated
- Retired
- Structurally unavailable
- Implemented but unreachable
- Duplicate
- Dead
- Information-only
- Tradeable

Re-prove the A0.3 setup-availability work, including honest retirement or gating of index setups that require unavailable inputs:

- `VOLUME_BREAKOUT` where cash-index volume is unavailable
- `MEAN_REVERSION` where session VWAP is unavailable
- `TREND_CONTINUATION` variants dependent on unavailable VWAP or insufficient confidence

The UI and API must explain setup unavailability. A structurally impossible setup must not silently return no signal.

Signal Classification

Verify separation among:

- `TRADEABLE_SIGNAL`
- `INFO_ONLY`
- `WATCHLIST_SETUP`
- Rejected candidate
- Pre-emission rejection
- Risk-blocked
- Data-blocked
- Cooldown-blocked
- Market-closed
- Provider-unavailable

Verify:

- Recovery veto
- Chase veto
- Anti-flip cooldown
- Duplicate-signal prevention
- signal expiry
- same-candle duplication
- same-direction repetition
- opposite-direction flip
- confidence thresholds
- explainability

- immutable signal plan

Previously accepted strategy thresholds and safeguards must not be changed merely to generate more trades. Audit whether they work as designed.

Option Contract Selection

Verify:

- CE versus PE direction
- expiry selection
- weekly/monthly expiry
- ATM/ITM/OTM logic
- strike interval
- lot size
- tick size
- instrument token
- exchange segment
- tradability
- liquidity
- bid/ask
- spread
- volume
- OI
- stale quote
- contract identity
- expiry-day rules
- missing contract handling

Entry, Stop, Target and Exit

Verify:

- Entry trigger
- limit/market modelling
- option-premium entry
- underlying-based versus premium-based stop
- stop-loss level
- target levels
- trailing stop
- time stop
- EOD exit
- expiry-day exit
- gap handling
- partial exit, if supported
- invalid tick handling
- best/worst price assumptions
- duplicate exit prevention
- closed-market exit policy
- shadow exit price
- missing-exit quote fallback

- stale-exit quote behaviour

Re-prove accepted work covering:

- Signal plan immutability
- Exit premium market shadow
- gross versus net P&L
- STT and charges
- MACD warm-up correctness

Candidate-to-Trade Conversion

Investigate why valid F&O candidates may result in zero opened trades.

Produce a full funnel:

- Universe evaluated
- data-ready
- setup detected
- candidate created
- candidate validated
- tradeable signal
- risk approved
- contract resolved
- quote accepted
- paper-trade attempted
- trade opened
- trade monitored
- trade exited

Provide counts and exact skip reasons for each stage.

A zero-trade day must explain whether the cause was:

- Genuine lack of setup
 - Kite unavailable
 - stale data
 - option chain unavailable
 - risk rejection
 - contract resolution failure
 - session failure
 - engine failure
 - scheduler failure
 - database failure
-

12. Known F&O Incidents Requiring Re-Verification

Do not assume these are fixed. Reproduce or prove their current state.

False “Market Is Closed” Message

Historically, `/options` could show “Market is closed” during IST market hours.

Audit:

- React Query/browser cache
- deprecated `marketState="closed"` fallbacks
- market-status API
- stale/missing/error state
- frontend derivation
- timezone
- service-worker cache

Required rule:

Only explicit, trustworthy `marketStatus.marketOpen === false` may produce a market-closed message. Missing, stale or failed status data must produce a neutral degraded/error state.

`/api/options/signals` Contract Failure

A historical HTTP 500 involved `contractInstrumentToken` and Zod/OpenAPI schema mismatch.

Verify:

- Database type
- domain type
- service output
- route serializer
- Zod contract
- OpenAPI response
- frontend type
- nullability
- numeric/string conversion

The endpoint must return a valid, honest response and include rich `marketStatus` and `setupState`.

Missing Execution Truth

Signal-history execution details were previously stripped by API contracts, causing false `NOT_CONFIRMED` messages.

Verify that:

- `execution` survives database → service → schema → API → frontend.
- `INFO_ONLY` correctly says no paper trade was expected.
- rejected signals display their real rejection reason.
- missing execution information is not represented as a failed trade.

Kite Session No-Signal Incident

A prior no-signal day occurred because Kite expired around 06:00 IST and login happened around 15:21 IST. Indices logged `PRE_EMISSION_REJECTED / no_live_kite_intraday`.

Verify:

- Pre-open readiness
- owner notification
- global degraded banner
- F&O page explanation
- scheduler behaviour
- no-signal reporting
- session-expiry observability

13. Swing Trading Engine Audit

Audit the complete swing lifecycle:

```
Universe
→ Fundamental/technical screening
→ Candidate ranking
→ Signal
→ Risk assessment
→ Staged order
→ Owner approval
→ Dry run/paper entry
→ Position monitoring
→ Stop/target/trailing exit
→ Ledger
→ P&L
→ Alert
→ Report
```

Verify:

- Universe completeness
- equity-only restrictions
- symbol resolution
- liquidity filters

- price filters
- volume filters
- delivery filters
- trend filters
- breakout logic
- pullback logic
- gap rules
- technical indicators
- fundamental inputs, if used
- corporate actions
- result reproducibility
- ranking stability
- duplicate candidates
- failed-symbol handling
- stale-data handling
- partial scan disclosure
- entry validity window
- staged-order expiry
- approval requirements
- dry-run behaviour
- risk blocking
- capital availability
- position limits
- concentration
- stop-loss
- target
- trailing stop
- gap-down handling
- market-closed monitoring
- exit availability
- charges
- reconciliation

Audit every staged-order status:

- STAGED
- APPROVAL_REQUIRED
- EXPIRED
- REJECTED
- APPROVED_DRY_RUN
- BLOCKED_BY_RISK

Ensure staged limit prices and Kite LTP are clearly distinguished.

Investigate historical runs where approximately 24–50 symbols failed.

Produce exact failure classifications:

- Instrument unresolved
- Provider missing
- API timeout

- Invalid candle history
 - Corporate-action inconsistency
 - Calculation failure
 - Database failure
 - Rate limit
 - stale data
 - eligibility rejection
-

14. Paper-Trading and Accounting Audit

Treat paper trading as a financial ledger, not a UI simulation.

Audit:

- Signal-to-trade linkage
- authorization
- entry insert
- exit insert
- position state
- quantity
- lot size
- average price
- partial fills, if supported
- capital reservation
- capital release
- realized P&L
- unrealized P&L
- gross P&L
- net P&L
- brokerage
- STT
- exchange charges
- GST
- SEBI charges
- stamp duty
- slippage
- cash ledger
- capital events
- duplicate mutation protection
- idempotency
- reconciliation
- audit trail
- manual intervention
- deletion protection
- timestamp integrity

Required accounting invariant:

```
Opening capital
+ deposits
- withdrawals
+ realized gross P&L
- total charges
= closing available capital
+ reserved capital adjustments
```

Reconcile:

- Trade-level P&L
- Day-level P&L
- Strategy-level P&L
- Index-level P&L
- Equity/F&O P&L
- Ledger balance
- Dashboard summary
- Daily report
- Exported report

Historical Stop-Ship Incident

Investigate the DLF AUTO paper trade recorded at approximately `18 Jul · 16:00:28`, including possible year/date corruption and market-closed entry.

Do not delete, auto-close or rewrite the suspicious row.

Preserve and trace:

- Original signal
- original timestamp
- authorization
- quote
- session state
- position insert
- ledger mutation
- capital mutation
- subsequent edits
- alerts
- job execution
- database timezone
- frontend formatting

The final position insert must be proven to require:

- Valid exchange session
- same-session authorization
- same-session quote
- fresh trade-grade Kite quote
- valid, unexpired signal

- idempotency
- correct instrument
- valid available capital

Required regression cases:

- Weekend
- Holiday
- Pre-open
- Exactly 09:15
- Exactly 15:30
- Post-close
- Special session
- Stale signal
- Expired signal
- Stale quote
- Prior-session quote
- Duplicate request
- Retry
- Direct API bypass
- Scheduler bypass
- Missing calendar
- Incorrect clock
- Database retry
- Concurrent inserts

Entry must be blocked outside the valid session. Legitimate risk-reducing exits must remain possible under an explicit, audited closed-market policy.

15. Calculation and Mathematical Accuracy Audit

Independently recompute all formulas.

General Market Calculations

Verify:

- Absolute change
- percentage change
- previous close
- gap
- returns
- CAGR
- XIRR
- moving averages
- VWAP
- RSI

- MACD
- ATR
- Bollinger Bands
- ADX
- Supertrend
- pivot levels
- support/resistance
- volume averages
- relative volume
- delivery percentage
- breadth
- sector returns
- beta
- volatility
- drawdown
- Sharpe/Sortino, if shown
- profit factor
- expectancy
- win rate
- risk-reward
- exposure

Derivatives Calculations

Verify:

- Futures basis
- annualized basis
- option intrinsic value
- time value
- moneyness
- implied volatility
- delta
- gamma
- theta
- vega
- rho
- PCR
- change-in-OI classification
- max pain
- expected move
- payoff
- breakeven
- margin references
- lot-based P&L
- premium turnover
- expiry handling

State all formulas, inputs, rounding rules and units.

Use independent test vectors, boundary cases and real exchange examples.

16. Database and Schema Audit

Audit every table, relation, constraint, migration and index.

Verify:

- Development/preview/production schema parity
- Migration history
- unapplied migrations
- schema drift
- Zod/TypeScript/database alignment
- primary keys
- foreign keys
- unique constraints
- check constraints
- not-null rules
- decimal precision
- timestamp type
- timezone
- indexes
- query plans
- retention
- partitioning, if used
- duplicate records
- orphaned records
- impossible states
- state transitions
- audit log
- soft deletion
- versioning
- backup
- restore testing

Pay special attention to:

- Signals
- Signal execution
- F&O paper trades
- Equity paper trades
- Capital events
- Positions
- Orders
- Quotes
- Candles
- Option-chain snapshots
- Provider health
- Job runs
- Alert delivery
- Instrument master

- Trading calendar
- Raw payload evidence

Check whether database constraints enforce critical invariants or whether they exist only in application code.

17. API and Contract Audit

For every API endpoint:

- Method
- Path
- Authentication
- Authorization
- Request schema
- Response schema
- Database access
- Provider calls
- Cache
- timeout
- retry
- rate limit
- idempotency
- pagination
- sorting
- validation
- status codes
- error structure
- provenance
- freshness
- logging
- correlation ID

Verify alignment across:

```
Database
↔ ORM
↔ Domain model
↔ Service
↔ Zod/OpenAPI
↔ JSON response
↔ Frontend query
↔ UI
```

Test:

- Valid response
- Empty result

- Missing provider
- timeout
- malformed provider response
- stale cache
- authentication failure
- authorization failure
- validation failure
- database failure
- partial result
- retry
- duplicate request
- high concurrency

No endpoint may return HTTP 200 with a misleading success payload when the business operation failed.

No expected absence of a trading signal should become HTTP 500.

18. Scheduler, Jobs and Operational Workflow Audit

Inventory all scheduled jobs and verify:

- Owner
- purpose
- frequency
- timezone
- trading-day dependency
- prerequisites
- idempotency
- distributed locking
- concurrency
- retry
- backoff
- timeout
- failure state
- last success
- next run
- alert on failure
- manual rerun
- safe recovery
- production enablement

Audit jobs for:

- Instrument-master refresh
- Exchange-calendar refresh
- Candle ingestion

- Candle backfill
- Option-chain capture
- Scanner runs
- Swing scans
- F&O signal generation
- Paper-trade monitoring
- Exit monitoring
- Daily reports
- Telegram alerts
- Cache refresh
- Database cleanup
- Health checks

Investigate historical candle-warehouse SQL failures, empty warehouse states and repeated job errors.

Prove that job failure does not silently produce apparently valid empty results.

19. Alerts and Notifications Audit

Audit Telegram and any other notification channel.

Verify:

- Correct recipient
- owner-only authorization
- trigger
- message content
- IST timestamp
- instrument
- signal class
- entry
- stop
- target
- confidence
- data source
- freshness
- trade/paper-trade status
- skip reason
- deduplication
- retry
- delivery receipt
- failure logging
- stale-message prevention
- environment separation

Ensure test and preview alerts cannot be mistaken for production alerts.

Do not send any message during the audit.

20. Security and Privacy Audit

Audit:

- Authentication
- session management
- owner-only authorization
- route protection
- object-level authorization
- API-key handling
- secret storage
- logs
- frontend bundles
- error messages
- database credentials
- webhook secrets
- broker tokens
- encryption
- CORS
- CSRF
- XSS
- SQL injection
- command injection
- SSRF
- insecure deserialization
- path traversal
- open redirects
- rate limiting
- brute-force protection
- dependency vulnerabilities
- source maps
- debug endpoints
- public data exposure
- admin endpoints
- backup exposure
- personally identifiable information
- audit logs

Search git history and deployed bundles for leaked credentials without printing any secret value into the report.

Report secrets only by name, location category and exposure status, always redacted.

21. Performance and Reliability Audit

Measure and report:

- Frontend load time

- API latency
- database latency
- provider latency
- scanner duration
- option-chain refresh rate
- WebSocket health
- memory
- CPU
- database connection pool
- queue depth
- cache hit rate
- bundle size
- long tasks
- network request count
- repeated polling
- N+1 queries
- timeout configuration
- retry storms
- rate-limit risk
- provider quota use
- graceful degradation

Test realistic cases:

- Market open
- high volatility
- provider outage
- partial provider outage
- Kite session expiry
- WebSocket disconnect
- database slowdown
- delayed quotes
- 1,000+ instrument universe
- simultaneous tab use
- browser refresh
- server restart
- deployment during market hours

22. UI, UX, Accessibility and Mobile Audit

Audit every screen at desktop, tablet and mobile widths.

Verify:

- Layout
- readability
- financial number formatting
- Indian number grouping
- positive/negative colours

- colour-blind accessibility
- keyboard navigation
- labels
- tooltips
- error messages
- loading skeletons
- empty states
- stale states
- touch targets
- horizontal scrolling
- tables
- charts
- modal behaviour
- form validation
- confirmations
- destructive actions
- accessibility names
- contrast
- focus order
- screen-reader compatibility

Ensure visual polish does not conceal missing or untrusted data.

23. Testing Audit

Run or inspect, where safely available:

- TypeScript typecheck
- Lint
- Unit tests
- Integration tests
- Contract tests
- API tests
- Database tests
- Migration tests
- End-to-end tests
- Browser tests
- Visual regression tests
- Accessibility tests
- Security tests
- Load tests
- Failure-injection tests
- Timezone tests
- Exchange-calendar tests
- Provider-failure tests
- Cache tests
- Concurrency tests
- Idempotency tests
- Accounting-invariant tests

- Production smoke tests

For every test suite report:

- Command
- Environment
- Pass/fail/skip count
- Duration
- Skipped tests
- flaky tests
- mocked dependencies
- real dependencies
- coverage
- what the suite proves
- what it does not prove

A skipped test is not a pass.

A mocked provider test does not prove live provider integration.

A local test does not prove production deployment.

Create a missing-test matrix linking each P0/P1 risk to required regression coverage.

24. Previous Accepted Work — Re-Prove and Do Not Regress

The project has previously reported or accepted work in the following areas. Treat these as claims requiring current verification:

- Canonical data backbone
- Candle source honesty
- Quote provenance and freshness
- Watchlist cleanup
- Portfolio resolution
- BSE pricing correction
- Charting source labels
- Trusted option-chain facade
- Trusted index-quote facade
- Paper-trading source safety
- Scanner/Deep Scan source honesty
- Sector and benchmark cleanup
- Telegram owner alerts
- Daily pre/post-market reports
- F&O signal-gap monitoring
- Swing staged-order alerts
- Signal plan immutability
- Exit premium market shadow

- Kite OI unit verification
- Gross/net F&O accounting and STT
- MACD warm-up
- A0.3 index-F&O setup availability
- API and UI disclosure of retired/unavailable setups

For each item, provide one of:

- Still correct and production verified
- Present but not production verified
- Partially regressed
- Fully regressed
- Superseded
- Unable to verify

Do not rewrite verified canonical modules unless the eventual remediation report proves a defect.

25. Areas Previously Deferred

Determine whether these remain intentionally deferred, have been implemented, or have created new risk:

- Pine import
- Drawing toolbar
- Chart-library replacement
- New indicators
- New F&O strategy logic
- Recovery/chase/anti-flip threshold changes
- P25 changes
- Guardrail loosening
- Dynamic lot-sizing changes
- Capital-ledger redesign
- Real broker execution

Do not label an intentionally deferred feature as a bug unless the existing UI or documentation claims it is available.

26. External Truth Validation

Validate important results against authoritative current sources.

Use primary sources wherever possible:

- NSE
- BSE
- SEBI
- Zerodha Kite official documentation

- Upstox official documentation
- IndianAPI official documentation
- official exchange circulars
- current contract/instrument masters
- official trading calendar
- official corporate-action files

For sampled instruments, compare at least:

- Instrument identity
- exchange
- symbol
- token
- LTP
- previous close
- OHLC
- timestamp
- volume
- OI
- lot size
- tick size
- expiry
- strike
- option type

Record the comparison timestamp and allowable tolerance.

Do not present a live-data comparison conducted after market close as proof of intraday behaviour.

27. Required Sampling

At minimum, audit representative instruments covering:

- NIFTY
- BANKNIFTY
- SENSEX
- Large-cap NSE equity
- Mid-cap equity
- Small-cap equity
- BSE-only or BSE-resolved holding
- ETF
- Equity with corporate action
- Illiquid option contract
- Near-expiry option
- Current weekly expiry
- Monthly expiry
- Missing/unresolved symbol
- Suspended or invalid symbol
- Expired derivative contract

Expand the sample wherever a defect appears systemic.

Sampling does not replace full code-path inventory or full database integrity checks.

28. Mandatory Deliverables

Produce one consolidated professional audit report with the following structure.

Deliverable 1 — Executive Verdict

State:

- Overall readiness
- Research-tool readiness
- Paper-trading readiness
- Unattended-paper-trading readiness
- Live-trading readiness
- Data-trust readiness
- Production reliability
- Security posture
- Whether real broker execution must remain disabled
- Top 10 risks
- Immediate stop-ship findings

Use explicit verdicts:

- PASS
- PASS WITH CONDITIONS
- FAIL
- NOT VERIFIED

Deliverable 2 — Complete System Inventory

Include every:

- Tab
- Route
- API
- Service
- Provider
- Job
- Table
- Strategy
- Signal type
- Trade state
- Alert
- Report
- Feature flag

Deliverable 3 — Tab-by-Tab Audit Matrix

For each tab provide:

Tab/ Feature	What Exists	What Works	What Is Wrong	Missing	Data Source	Production Status	Evidence
-----------------	----------------	---------------	------------------	---------	----------------	----------------------	----------

Deliverable 4 — Master Issue Register

Use:

ID	Severity	Module	Finding	Evidence	Root Cause	Impact	Recommended Fix	Required Test	Status
----	----------	--------	---------	----------	---------------	--------	--------------------	------------------	--------

Include all P0–P4 items without duplicates.

Deliverable 5 — Data-Lineage and Provider Matrix

Map every important data point from provider to UI and trading consumer.

Deliverable 6 — Calculation Verification Workbook/Table

For each calculation provide:

- Formula
- Code location
- Independent result
- Application result
- Difference
- rounding
- unit
- verdict

Deliverable 7 — F&O Audit

Include:

- Setup inventory
- setup availability
- signal funnel
- rejection reasons
- contract selection
- entry/exit validation
- paper-trade reconciliation
- index-wise findings
- date-wise findings
- trade-wise findings

- current blockers

Deliverable 8 — Swing Audit

Include:

- Scanner funnel
- failed-symbol analysis
- candidate ranking
- staged-order state machine
- approval controls
- trade lifecycle
- risk controls
- reconciliation
- current blockers

Deliverable 9 — Paper-Trading and Capital Reconciliation

Include:

- Every open trade
- every closed trade
- orphaned positions
- duplicate records
- out-of-session entries
- ledger reconciliation
- capital drift
- charge calculation
- unexplained differences
- historical DLF incident trail

Deliverable 10 — Production vs Repository Parity

Report:

- Repository commit
- production commit
- schema parity
- environment parity
- feature-flag parity
- API parity
- UI parity
- missing deployment changes
- unidentified production changes

Deliverable 11 — Security Report

Include vulnerabilities, public exposure, owner-only access verification and secret-handling findings.

Deliverable 12 — Testing Gap Report

Map every critical requirement to:

- Existing tests
- result
- missing tests
- mocked versus real evidence
- production verification requirement

Deliverable 13 — Prioritised Remediation Roadmap

Separate:

Immediate P0 containment

Actions needed before further paper trading.

P1 correctness and reliability

Data, trading, accounting and security corrections.

P2 operational improvements

Performance, observability, reporting and UX.

P3 cleanup

Maintainability and documentation.

For every task provide:

- Scope
- dependencies
- affected files/modules
- risk
- estimated effort
- acceptance tests
- production-verification method
- rollback plan

Do not group unrelated issues into one vague task.

Deliverable 14 — Final Acceptance Manifest

Provide a checklist that can later be used to certify remediation.

Each item must specify:

- Requirement

- proof command/test
 - expected result
 - production evidence
 - pass/fail
 - remaining limitation
-

29. Mandatory Final Questions the Audit Must Answer

The report is incomplete unless it clearly answers all of the following:

1. Is production running the audited code?
2. Is there one canonical data backbone?
3. Does every critical number have traceable provenance?
4. Are any values fabricated, stale, delayed, synthetic or mislabelled?
5. Do all tabs use the same authoritative data point for the same instrument and time?
6. Can missing data become zero or a valid-looking value?
7. Can provider failure be mistaken for “market closed” or “no opportunity”?
8. Can a trade open outside a valid exchange session?
9. Can a trade open using a stale or prior-session quote?
10. Can a signal be duplicated or retrospectively changed?
11. Can paper capital mutate twice?
12. Does the ledger reconcile exactly?
13. Are P&L and charges correct?
14. Are F&O option selection, lots, stops and targets correct?
15. Why do valid candidates become zero trades?
16. Are skip reasons complete and visible?
17. Are retired/unavailable setups disclosed honestly?
18. Is swing scanning complete and reproducible?
19. Why do swing symbols fail?
20. Are backtest premiums real, modelled or synthetic?
21. Is there look-ahead or survivorship bias?
22. Are market calendar and IST boundaries correct?
23. Are caches capable of overriding live truth?
24. Are database, API and frontend schemas aligned?
25. Are preview and production databases aligned?
26. Are scheduled jobs actually executing successfully?
27. Are alerts correct, timely and deduplicated?
28. Is owner-only authorization enforced everywhere?
29. Are credentials and broker tokens secure?
30. Can live broker execution occur accidentally?
31. What is correct and should remain untouched?
32. What is broken?
33. What is partially implemented?
34. What is missing?
35. What is duplicated?
36. What is suspicious?
37. What is unsafe?

- 38. What cannot be verified?
 - 39. What must be fixed first?
 - 40. What exact evidence will prove the project is professionally ready?
-

30. Auditor Conduct Rules

- Be direct and evidence-based.
 - Do not flatter the implementation.
 - Do not exaggerate defects.
 - Do not hide uncertainty.
 - Do not report assumptions as facts.
 - Do not fabricate test results.
 - Do not claim “production verified” without production evidence.
 - Do not treat UI rendering as functional correctness.
 - Do not treat a passing build as trading correctness.
 - Do not recommend loosening controls to increase signal counts.
 - Do not remove safety rules without quantitative evidence.
 - Do not change trading logic during the audit.
 - Do not delete polluted records.
 - Do not expose secrets.
 - Do not execute real orders.
 - Do not omit minor findings merely because critical findings exist.
 - Remove duplicate findings while preserving all distinct evidence.
 - Preserve exact historical and production evidence.
 - Cite exact code and authoritative external sources.
 - Explain all financial consequences in plain language.
 - Report both strengths and weaknesses.
 - Clearly distinguish bugs from future enhancements.
 - Clearly distinguish implemented code from proven behaviour.
 - Clearly distinguish data problems from genuine market conditions.
-

31. Completion Standard

Do not declare the audit complete until:

- Every repository module has been inventoried.
- Every frontend route has been audited.
- Every backend route has been classified.
- Every visible tab has been tested.
- Every provider path has been mapped.
- Every scheduled job has been reviewed.
- Every trading lifecycle has been traced.
- Every financial calculation has been independently checked.
- Every database mutation path has been audited.
- Every known historical incident has been re-verified.
- Development, preview and production differences are documented.
- All issues are included in one deduplicated register.

- All conclusions contain evidence or are marked not verified.
- A prioritised remediation roadmap is supplied.
- A final acceptance manifest is supplied.

The required outcome is not a generic software review. It is a complete, professional, forensic audit establishing whether NSC Scanner can be trusted as an Indian-market analysis and supervised paper-trading platform, exactly what remains unsafe or incomplete, and what evidence will be required before any future expansion toward live execution.